Alternative Assessment Tool(AAT) Report on

"Design and verification of Combinational and Sequential Digital Modules using Verilog HDL"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL LOGIC DESIGN

[Course code: 23EC5PE1AD]

SUBMITTED BY

R Dhruthi Veda Samhitha **1BM24EC157**

Course Instructor

Prof. Ajaykumar Devarapalli

Department of Electronics and Communications

BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year
2026 – 2027

TABLE OF CONTENTS

Sl. No.	Experiment Title	Page No.
1	Arithmetic Logic Unit (ALU)	1-2
2	Comparator	2-4
3	4-Bit Parity Generator and Checker	4-6
4	4-Bit Barrel Shifter	7-8
5	Timer Module	8-10
6	Ring Counter	10-11
7	Johnson Counter	12-13
8	Synchronous and Asynchronous Counters	13-19
9	Shift Register	19-21
10	Divide-by-N Counter	22-23
11	Mealy and Moore FSM	23-25
12	Random Sequence Generator	26-28

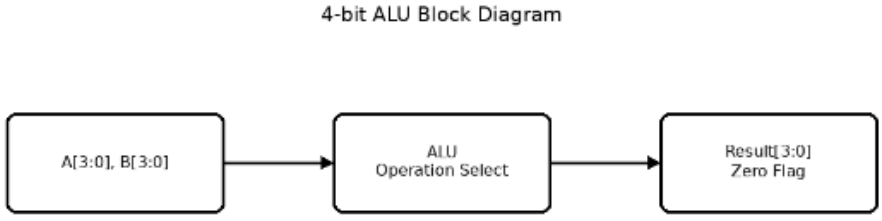
Experiment 1: 4-bit Arithmetic Logic Unit (ALU)

Design:

Truth Table / Operation Table

sel	Operation	Expression
000	Addition	A + B
001	Subtraction	A - B
010	Bitwise AND	A & B
011	Bitwise OR	A | B
100	Bitwise XOR	A ^ B
others	Default	0

Block Diagram:



Verilog HDL Design Code:

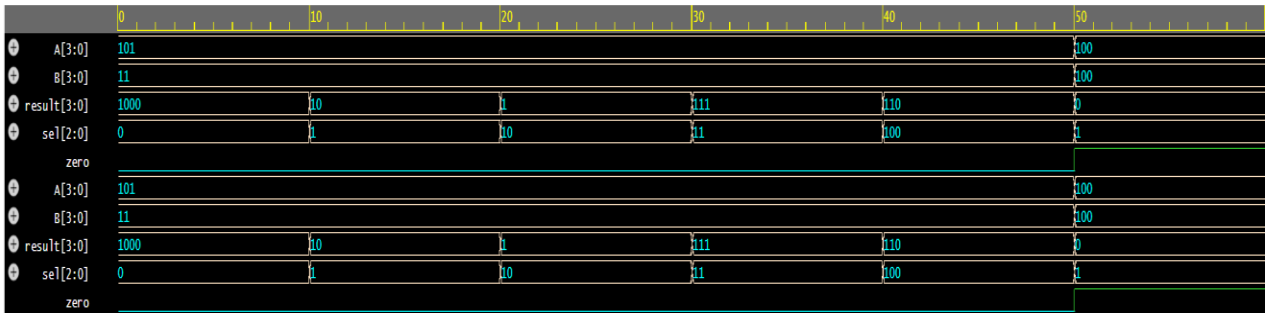
```
module alu (
   input  [3:0] A, B,
   input  [2:0] sel,
   output reg [3:0] result,
   output zero
);
always @(*) begin
   case (sel)
      3'b000: result = A + B;
      3'b001: result = A - B;
      3'b010: result = A & B;
      3'b011: result = A | B;
      3'b100: result = A ^ B;
      default: result = 4'b0000;
   endcase
end
assign zero = (result == 4'b0000);
endmodule
```

Testbench Code:

```
module alu_tb;
reg [3:0] A, B;
reg [2:0] sel;
wire [3:0] result;
```

```
wire zero;
alu dut(.A(A), .B(B), .sel(sel), .result(result), .zero(zero));
  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, alu_tb);
end
initial begin
    A=4'd5; B=4'd3; sel=3'b000; #10;
    sel=3'b001; #10;
    sel=3'b010; #10;
    sel=3'b011; #10;
    sel=3'b100; #10;
    A=4'd4; B=4'd4; sel=3'b001; #10;
    $finish;
end
initial begin
    $monitor("t=%0t A=%d B=%d sel=%b result=%d zero=%b",
            $time,A,B,sel,result,zero);
end
endmodule
```

Waveform:



Output:

t=0 A= 5 B= 3 sel=000 result= 8 zero=0
t=10 A= 5 B= 3 sel=001 result= 2 zero=0
t=20 A= 5 B= 3 sel=010 result= 1 zero=0
t=30 A= 5 B= 3 sel=011 result= 7 zero=0
t=40 A= 5 B= 3 sel=100 result= 6 zero=0
t=50 A= 4 B= 4 sel=001 result= 0 zero=1

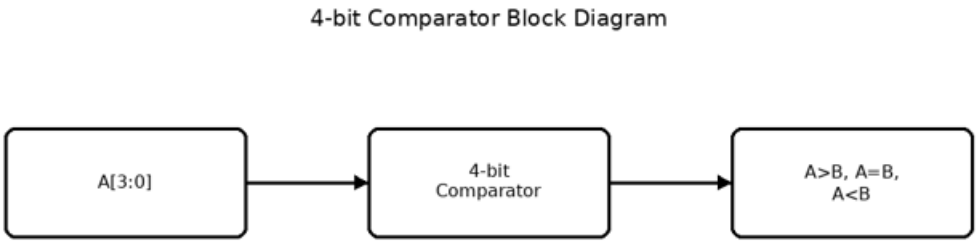
Experiment 2: 4-bit Magnitude Comparator

Design:

Truth Table / Operation Table

Condition	A_gt_B	A_eq_B	A_lt_B
A > B	1	0	0
A = B	0	1	0
A < B	0	0	1

Block Diagram



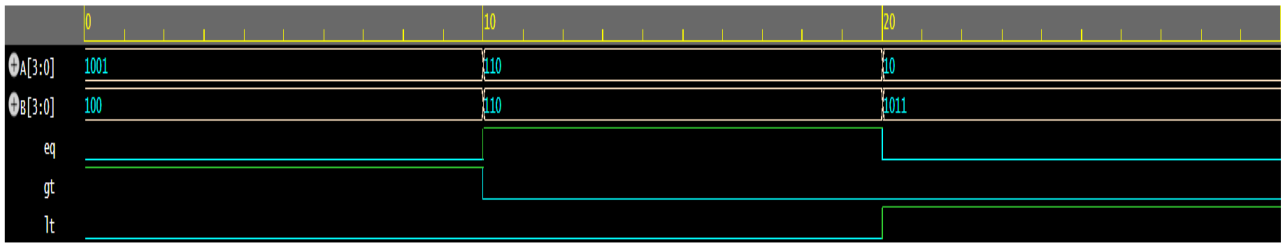
Verilog HDL Design Code:

```
module comparator #(parameter N=4) (
   input  [N-1:0] A, B,
   output reg A_gt_B, A_eq_B, A_lt_B
);
always @(*) begin
   A_gt_B = 1'b0;
   A_eq_B = 1'b0;
   A_lt_B = 1'b0;
   if (A > B) A_gt_B = 1'b1;
   else if (A == B) A_eq_B = 1'b1;
   else A_lt_B = 1'b1;
end
endmodule
```

Testbench Code:

```
module comparator_tb;
reg [3:0] A, B;
wire gt, eq, lt;
comparator #(4) dut(.A(A), .B(B),
   .A_gt_B(gt), .A_eq_B(eq), .A_lt_B(lt));
  initial begin
   $dumpfile("dump.vcd");
   $dumpvars(0,comparator_tb );
end
initial begin
   A=4'd9; B=4'd4; #10;
   A=4'd6; B=4'd6; #10;
   A=4'd2; B=4'd11; #10;
   $finish;
end
initial $monitor("A=%d B=%d gt=%b eq=%b lt=%b",A,B,gt,eq,lt);
endmodule
```

Waveforms:



Output:
A= 9 B= 4 gt=1 eq=0 lt=0
A= 6 B= 6 gt=0 eq=1 lt=0
A= 2 B=11 gt=0 eq=0 lt=1

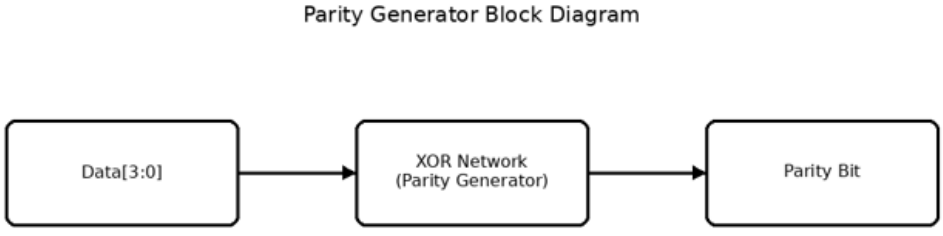
Experiment 3: 4-bit Even Parity Generator and Checker

Design:

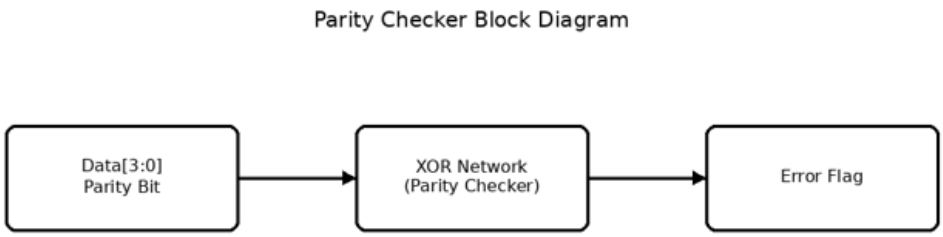
Truth Table / Operation Table

Data D[3:0]	Number of 1s	Even parity bit P
0000	0	0
0001	1	1
0011	2	0
0111	3	1
1111	4	0

Block Diagram



Checker block diagram:



Verilog HDL Design Code

```
module parity_generator (
   input [3:0] data,
   output parity
);
   // Generate even parity
   assign parity = ^data;
endmodule


module parity_checker (
   input [3:0] data,
   input parity,
   output error
);
   // Error is 1 if parity does not match
   assign error = ^{data, parity};
endmodule
```

Testbench:

```
module parity_tb;

   reg [3:0] data;
   reg parity_in;

   wire parity_out;
   wire error;

   parity_generator gen (
      .data(data),
      .parity(parity_out)
   );

   parity_checker chk (
      .data(data),
      .parity(parity_in),
      .error(error)
   );

   initial begin
      $dumpfile("dump.vcd");
      $dumpvars(0, parity_tb);

      $monitor("Time=%0t data=%b generated_parity=%b received_parity=%b error=%b",
            $time, data, parity_out, parity_in, error);

      // Test 1: No 1s
      data = 4'b0000;
      #10 parity_in = parity_out;
```

```
        #10;

        // Test 2: One 1
        data = 4'b0001;
        #10 parity_in = parity_out;
        #10;

        // Test 3: Two 1s
        data = 4'b0011;
        #10 parity_in = parity_out;
        #10;

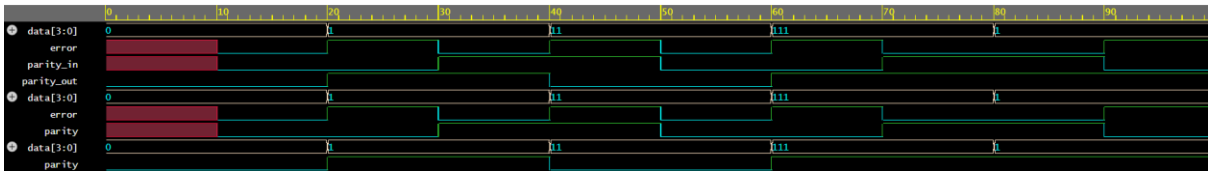
        // Test 4: Three 1s
        data = 4'b0111;
        #10 parity_in = parity_out;
        #10;

        // Test 5: Introduce a parity error
        data = 4'b0001;
        #10 parity_in = ~parity_out;
        #10;

        $finish;
    end

endmodule
```

Waveform:



Output:

```
Time=0 data=0000 generated_parity=0 received_parity=x error=x
Time=10 data=0000 generated_parity=0 received_parity=0 error=0
Time=20 data=0001 generated_parity=1 received_parity=0 error=1
Time=30 data=0001 generated_parity=1 received_parity=1 error=0
Time=40 data=0011 generated_parity=0 received_parity=1 error=1
Time=50 data=0011 generated_parity=0 received_parity=0 error=0
Time=60 data=0111 generated_parity=1 received_parity=0 error=1
Time=70 data=0111 generated_parity=1 received_parity=1 error=0
Time=80 data=0001 generated_parity=1 received_parity=1 error=0
Time=90 data=0001 generated_parity=1 received_parity=0 error=1
```

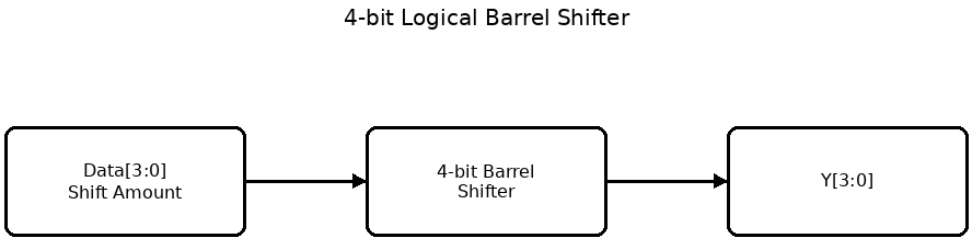
Experiment 4: 4-bit Logical Barrel Shifter

Design:

Truth Table / Operation Table

dir	shamt	Operation
0	00	No shift
0	01	Left by 1
0	10	Left by 2
0	11	Left by 3
1	00	No shift
1	01	Right by 1
1	10	Right by 2
1	11	Right by 3

Block Diagram



Verilog HDL Design Code

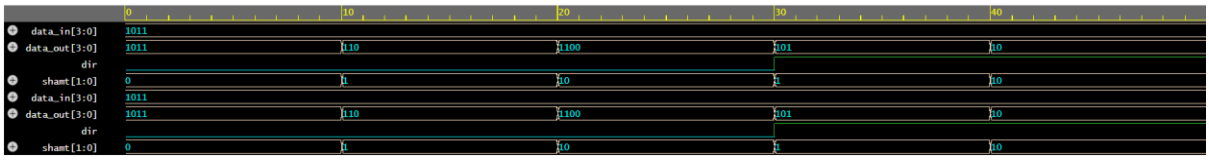
```
module barrel_shifter (
   input [3:0] data_in,
   input [1:0] shamt,
   input dir,            // 0=left, 1=right
   output reg [3:0] data_out
);
always @(*) begin
   if (dir == 1'b0)
      data_out = data_in << shamt;
   else
      data_out = data_in >> shamt;
end
endmodule
```

Testbench:

```
module barrel_shifter_tb;
reg [3:0] data_in;
reg [1:0] shamt;
reg dir;
wire [3:0] data_out;
```

```
barrel_shifter dut(.data_in(data_in), .shamt(shamt),
                   .dir(dir), .data_out(data_out));
  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, barrel_shifter_tb);
end
initial begin
    data_in=4'b1011; dir=0; shamt=0; #10;
    shamt=1; #10;
    shamt=2; #10;
    dir=1; shamt=1; #10;
    shamt=2; #10;
    $finish;
end
initial $monitor("data=%b dir=%b shamt=%d output=%b",
                 data_in,dir,shamt,data_out);
endmodule
```

Waveform:



Output:

```
data=1011 dir=0 shamt=0 output=1011
data=1011 dir=0 shamt=1 output=0110
data=1011 dir=0 shamt=2 output=1100
data=1011 dir=1 shamt=1 output=0101
data=1011 dir=1 shamt=2 output=0010
```

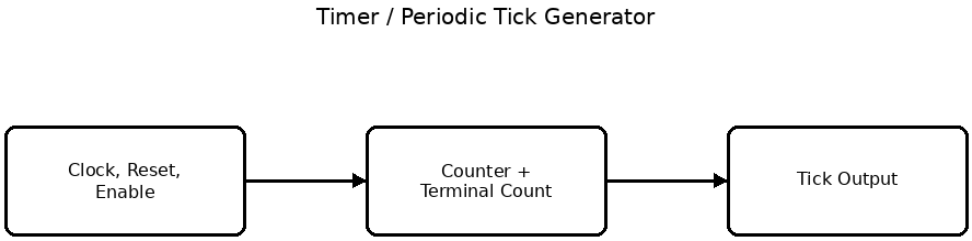
Experiment 5: Timer Module

Design:

Truth Table / Operation Table

Signal/parameter	Function
clk	System clock
reset	Synchronous active-high reset
enable	Enables counting
DIVISOR	Number of clock cycles per tick
tick	One-cycle pulse at terminal count

Block Diagram



Verilog HDL Design Code:

```
module timer #(
   parameter DIVISOR = 10
) (
   input clk, reset, enable,
   output reg tick
);
localparam CW = (DIVISOR <= 2) ? 1 : $clog2(DIVISOR);
reg [CW-1:0] count;
always @(posedge clk) begin
   if (reset) begin
      count <= 0;
      tick <= 0;
   end else if (!enable) begin
      count <= 0;
      tick <= 0;
   end else if (count == DIVISOR-1) begin
      count <= 0;
      tick <= 1;
   end else begin
      count <= count + 1'b1;
      tick <= 0;
   end
end
endmodule
```

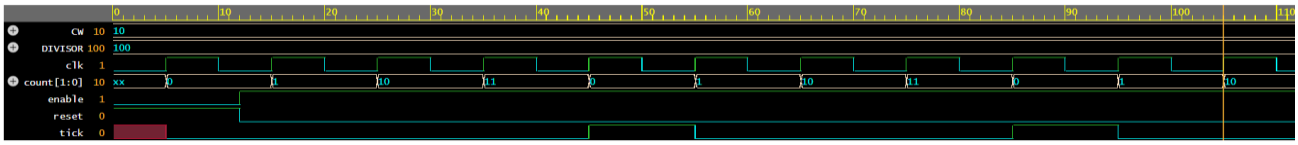
Testbench Code:

```
module timer_tb;
reg clk, reset, enable;
wire tick;
timer #(4) dut(.clk(clk), .reset(reset), .enable(enable), .tick(tick));
   initial begin
   $dumpfile("dump.vcd");
     $dumpvars(0, timer_tb);
end

always #5 clk = ~clk;
initial begin
```

```
    clk=0; reset=1; enable=0;
    #12; reset=0; enable=1;
    #100; $finish;
end
initial $monitor("t=%0t reset=%b enable=%b tick=%b",
                $time,reset,enable,tick);
endmodule
```

Waveforms:



Output:

t=0 reset=1 enable=0 tick=x
t=5 reset=1 enable=0 tick=0
t=12 reset=0 enable=1 tick=0
t=45 reset=0 enable=1 tick=1
t=55 reset=0 enable=1 tick=0
t=85 reset=0 enable=1 tick=1
t=95 reset=0 enable=1 tick=0

Experiment 6: 4-bit Ring Counter

Design:

Truth Table / Operation Table

Clock after reset	Q[3:0]
0	0001
1	0010
2	0100
3	1000
4	0001

Block Diagram



Verilog HDL Design Code

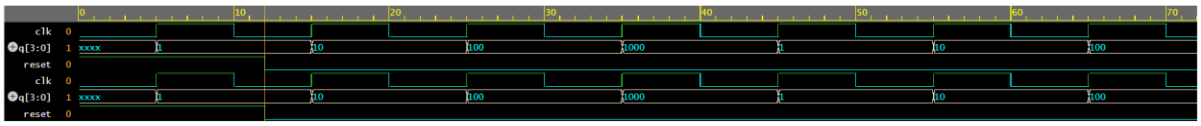
```
module ring_counter (
   input clk, reset,
   output reg [3:0] q
);
always @(posedge clk) begin
   if (reset) q <= 4'b0001;
   else q <= {q[2:0], q[3]};
end
endmodule
```

Testbench Code:

```
module ring_counter_tb;
reg clk, reset;
wire [3:0] q;
ring_counter dut(.clk(clk), .reset(reset), .q(q));
  initial begin
    $dumpfile("dump.vcd");
      $dumpvars(0, ring_counter_tb);
end

always #5 clk=~clk;
initial begin
   clk=0; reset=1; #12;
   reset=0; #60;
   $finish;
end
initial $monitor("t=%0t reset=%b q=%b",$time,reset,q);
endmodule
```

Waveforms:



Output:

```
t=0 reset=1 q=xxxx
t=5 reset=1 q=0001
t=12 reset=0 q=0001
t=15 reset=0 q=0010
t=25 reset=0 q=0100
t=35 reset=0 q=1000
t=45 reset=0 q=0001
t=55 reset=0 q=0010
t=65 reset=0 q=0100
```

Experiment 7: 4-bit Johnson Counter

Design:

Truth Table / Operation Table

Clock after reset	Q[3:0]
0	0000
1	0001
2	0011
3	0111
4	1111
5	1110
6	1100
7	1000
8	0000

Block Diagram



Verilog HDL Design Code

```
module johnson_counter (
   input clk, reset,
   output reg [3:0] q
);
always @(posedge clk) begin
   if (reset) q <= 4'b0000;
   else q <= {q[2:0], ~q[3]};
end
endmodule
```

Testbench Code:

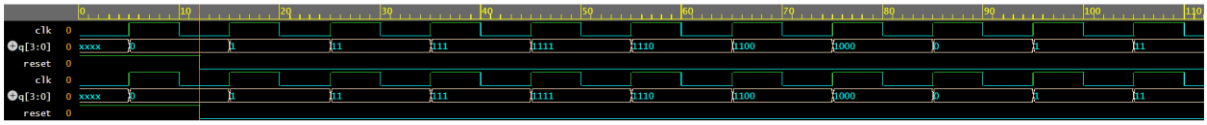
```
module johnson_counter_tb;
reg clk, reset;
wire [3:0] q;
johnson_counter dut(.clk(clk), .reset(reset), .q(q));

   initial begin
   $dumpfile("dump.vcd");
```

```
    $dumpvars(0, johnson_counter_tb);
end

always #5 clk=~clk;
initial begin
   clk=0; reset=1; #12;
   reset=0; #100;
   $finish;
end
initial $monitor("t=%0t reset=%b q=%b",$time,reset,q);
endmodule
```

Waveforms:



Output:

t=0 reset=1 q=xxxx
t=5 reset=1 q=0000
t=12 reset=0 q=0000
t=15 reset=0 q=0001
t=25 reset=0 q=0011
t=35 reset=0 q=0111
t=45 reset=0 q=1111
t=55 reset=0 q=1110
t=65 reset=0 q=1100
t=75 reset=0 q=1000
t=85 reset=0 q=0000
t=95 reset=0 q=0001
t=105 reset=0 q=0011

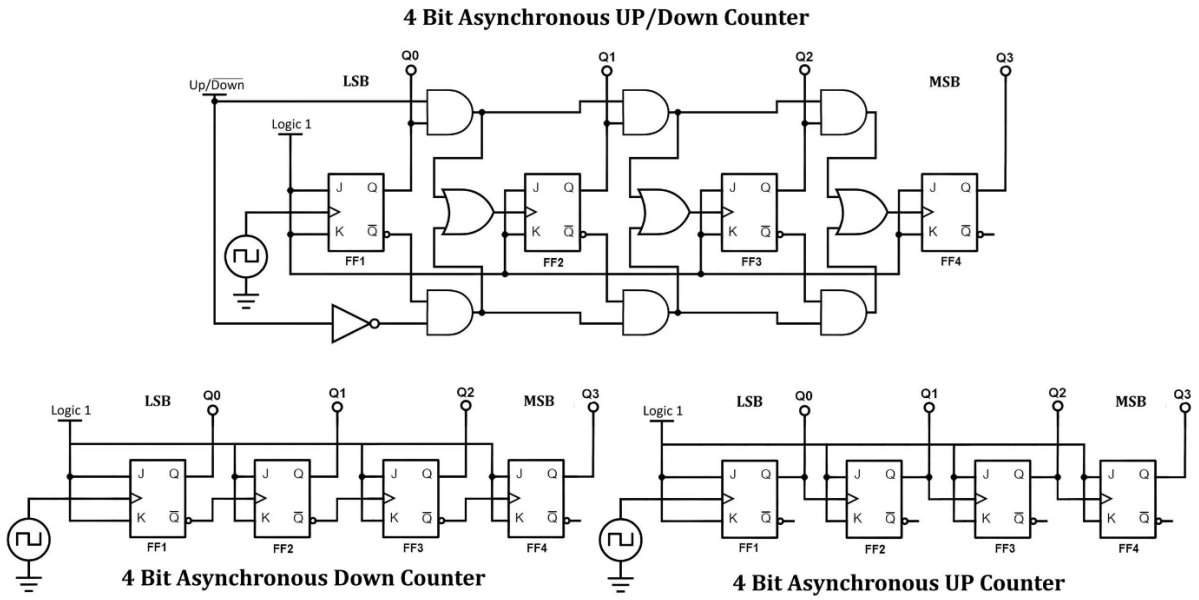
Experiment 9: 4-bit Asynchronous&Synchronous (Ripple) Up-down Counter

Design:

Truth Table / Operation Table

Feature	Synchronous counter	Asynchronous counter
Clock input	Same clock to all stages	Clock ripples between stages
Propagation	Stages update together	Stage-to-stage delay
Typical use	Higher-speed counting	Simple frequency division

a) Asynchronous counter:
Block Diagram:



Verilog HDL code:

```
module JK_flipflop (
   input clk,
   input rst_n,
   input j, k,
   output reg q,
   output q_bar
);

   always @(posedge clk or negedge rst_n) begin
      if (!rst_n)
         q <= 1'b0;
      else begin
         case ({j, k})
            2'b00: q <= q;
            2'b01: q <= 1'b0;
            2'b10: q <= 1'b1;
            2'b11: q <= ~q;
         endcase
      end
   end

   assign q_bar = ~q;

endmodule


module updown_selector (
```

```
    input q,
    input q_bar,
    input up,
    output nclk
);

    assign nclk = up ? q_bar : q;

endmodule


module asynchronous_counter #(
    parameter SIZE = 4
)(
    input clk,
    input rst_n,
    input up,
    output [SIZE-1:0] q,
    output [SIZE-1:0] q_bar
);

    wire [SIZE-2:0] nclk;
    genvar g;

    JK_flipflop jk0(
        clk, rst_n, 1'b1, 1'b1, q[0], q_bar[0]
    );

    generate
        for (g = 1; g < SIZE; g = g + 1) begin : counter_stages
            updown_selector ud(
                q[g-1], q_bar[g-1], up, nclk[g-1]
            );

            JK_flipflop jk(
                nclk[g-1], rst_n, 1'b1, 1'b1,
                q[g], q_bar[g]
            );
        end
    endgenerate

endmodule
```

Testbench:

```
module tb;

    reg clk, rst_n, up;
    wire [3:0] q, q_bar;
```

```
    asynchronous_counter dut(clk, rst_n, up, q, q_bar);
    initial begin
    $dumpfile("dump.vcd");
      $dumpvars(0, tb);
end


    always #5 clk = ~clk;

    initial begin
        $monitor("Time=%0t up=%b q=%b", $time, up, q);

        clk = 0;
        rst_n = 0;
        up = 1;

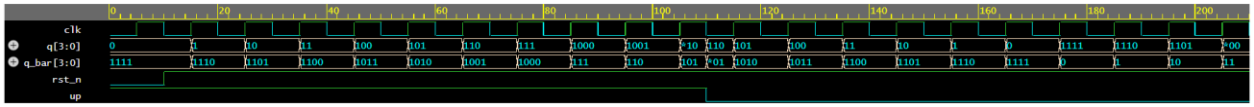
        #10 rst_n = 1;

        #100 up = 0;

        #100 $finish;
    end

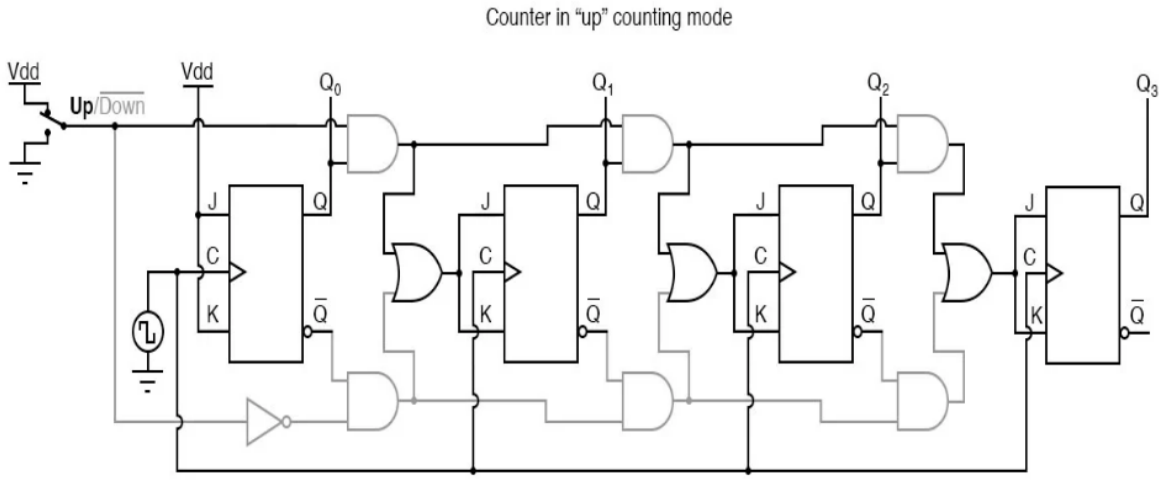
endmodule
```

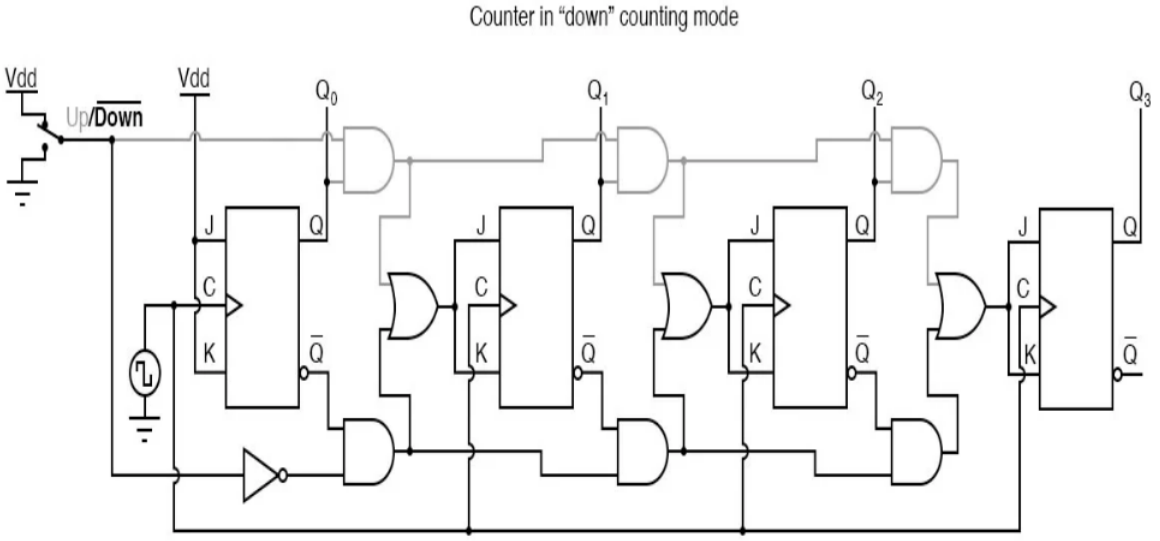
Waveforms:



b) Synchronous Counter

Block Diagram:





Verilog HDL code:

```
module JK_flipflop(
   input clk, rst_n, j, k,
   output reg q,
   output q_bar
);

   always @(posedge clk or negedge rst_n) begin
      if (!rst_n)
         q <= 0;
      else begin
         case ({j,k})
            2'b00: q <= q;
            2'b01: q <= 0;
            2'b10: q <= 1;
            2'b11: q <= ~q;
         endcase
      end
   end

   assign q_bar = ~q;

endmodule


module synchronous_counter(
   input clk, rst_n, up,
   output [3:0] q,
   output [3:0] q_bar
);

   // All flip-flops use the same clock
```

```
    JK_flipflop jk0(clk, rst_n, 1'b1, 1'b1, q[0], q_bar[0]);

    JK_flipflop jk1(clk, rst_n,
                up ? q[0] : q_bar[0],
                up ? q[0] : q_bar[0],
                q[1], q_bar[1]);

    JK_flipflop jk2(clk, rst_n,
                up ? (q[1] & q[0]) : (q_bar[1] & q_bar[0]),
                up ? (q[1] & q[0]) : (q_bar[1] & q_bar[0]),
                q[2], q_bar[2]);

    JK_flipflop jk3(clk, rst_n,
                up ? (q[2] & q[1] & q[0]) :
                    (q_bar[2] & q_bar[1] & q_bar[0]),
                up ? (q[2] & q[1] & q[0]) :
                    (q_bar[2] & q_bar[1] & q_bar[0]),
                q[3], q_bar[3]);

endmodule
```

Testbench Code:

```
module tb;

    reg clk, rst_n, up;
    wire [3:0] q, q_bar;

    synchronous_counter dut(clk, rst_n, up, q, q_bar);
 initial begin
    $dumpfile("dump.vcd");
      $dumpvars(0, tb);
end
    always #5 clk = ~clk;

    initial begin
        $monitor("Time=%0t | up=%b | q=%b",
                $time, up, q);

        clk = 0;
        rst_n = 0;
        up = 1;

        #10 rst_n = 1;

        // Up counting
        #100;

        // Down counting
        up = 0;
```

```
        #100;

        $finish;
    end

endmodule
```

Waveforms:

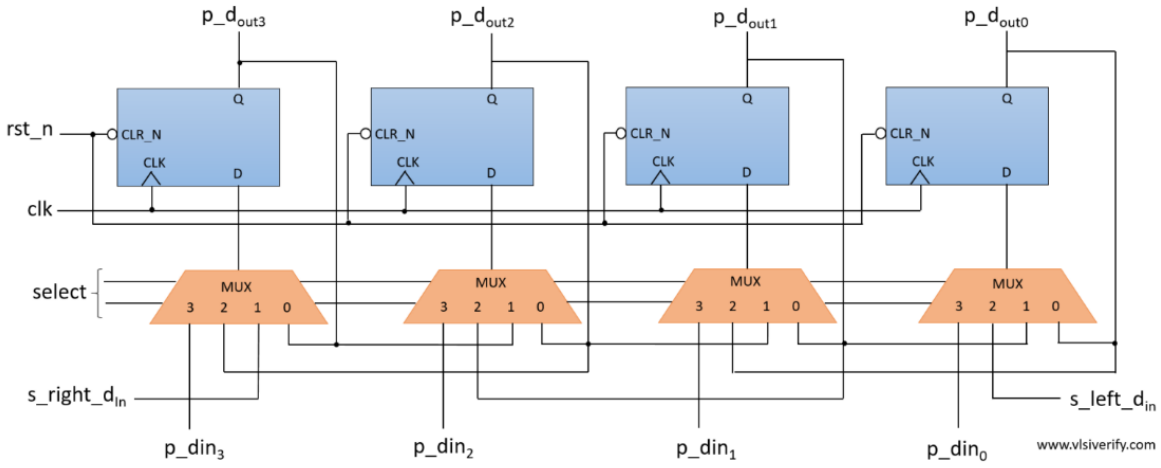


Experiment 9: Universal Shift Register

Design:

Mode control (select)		Operation
0	0	No change
0	1	Shift right
1	0	Shift left
1	1	Parallel load

Block Diagram:



Verilog HDL Code:

```
module universal_shift_reg(
  input clk, rst_n,
  input [1:0] select, // select operation
  input [3:0] p_din,  // parallel data in
  input s_left_din,   // serial left data in
  input s_right_din,  // serial right data in
```

```
  output reg [3:0] p_dout, //parallel data out
  output s_left_dout, // serial left data out
  output s_right_dout // serial right data out
);
  always@(posedge clk) begin
    if(!rst_n) p_dout <= 0;
    else begin
      case(select)
        2'h1: p_dout <= {s_right_din,p_dout[3:1]}; // Right Shift
        2'h2: p_dout <= {p_dout[2:0],s_left_din};  // Left Shift
        2'h3: p_dout <= p_din; // Parallel in - Parallel out
        default: p_dout <= p_dout; // Do nothing
      endcase
    end
  end
  assign s_left_dout = p_dout[0];
  assign s_right_dout = p_dout[3];
endmodule
```

Testbench Code:

```
module TB;
  reg clk, rst_n;
  reg [1:0] select;
  reg [3:0] p_din;
  reg s_left_din, s_right_din;
  wire [3:0] p_dout; //parallel data out
  wire s_left_dout, s_right_dout;

  universal_shift_reg usr(clk, rst_n, select, p_din, s_left_din, s_right_din, p_dout, s_left_dout,
s_right_dout);
  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, TB);
end

  always #2 clk = ~clk;
  initial begin
    $monitor("select=%b, p_din=%b, s_left_din=%b, s_right_din=%b --> p_dout = %b,
s_left_dout = %b, s_right_dout = %b",select, p_din, s_left_din, s_right_din, p_dout,
s_left_dout, s_right_dout);
    clk = 0; rst_n = 0;
    #3 rst_n = 1;

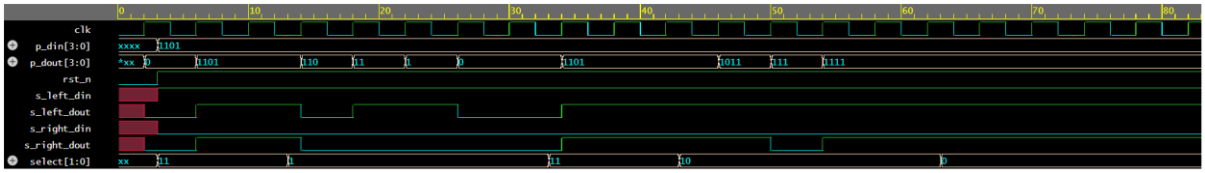
    p_din = 4'b1101;
    s_left_din = 1'b1;
    s_right_din = 1'b0;

    select = 2'h3; #10;
    select = 2'h1; #20;
```

```
    p_din = 4'b1101;
    select = 2'h3; #10;
    select = 2'h2; #20;
    select = 2'h0; #20;

    $finish;
  end
endmodule
```

Waveforms:



Output:

select=xx, p_din=xxxx, s_left_din=x, s_right_din=x --> p_dout = xxxx, s_left_dout = x, s_right_dout = x
select=xx, p_din=xxxx, s_left_din=x, s_right_din=x --> p_dout = 0000, s_left_dout = 0, s_right_dout = 0
select=11, p_din=1101, s_left_din=1, s_right_din=0 --> p_dout = 0000, s_left_dout = 0, s_right_dout = 0
select=11, p_din=1101, s_left_din=1, s_right_din=0 --> p_dout = 1101, s_left_dout = 1, s_right_dout = 1
select=01, p_din=1101, s_left_din=1, s_right_din=0 --> p_dout = 1101, s_left_dout = 1, s_right_dout = 1
select=01, p_din=1101, s_left_din=1, s_right_din=0 --> p_dout = 0110, s_left_dout = 0, s_right_dout = 0
select=01, p_din=1101, s_left_din=1, s_right_din=0 --> p_dout = 0011, s_left_dout = 1, s_right_dout = 0
select=01, p_din=1101, s_left_din=1, s_right_din=0 --> p_dout = 0001, s_left_dout = 1, s_right_dout = 0
select=01, p_din=1101, s_left_din=1, s_right_din=0 --> p_dout = 0000, s_left_dout = 0, s_right_dout = 0
select=11, p_din=1101, s_left_din=1, s_right_din=0 --> p_dout = 0000, s_left_dout = 0, s_right_dout = 0
select=11, p_din=1101, s_left_din=1, s_right_din=0 --> p_dout = 1101, s_left_dout = 1, s_right_dout = 1
select=10, p_din=1101, s_left_din=1, s_right_din=0 --> p_dout = 1101, s_left_dout = 1, s_right_dout = 1
select=10, p_din=1101, s_left_din=1, s_right_din=0 --> p_dout = 1011, s_left_dout = 1, s_right_dout = 1
select=10, p_din=1101, s_left_din=1, s_right_din=0 --> p_dout = 0111, s_left_dout = 1, s_right_dout = 0
select=10, p_din=1101, s_left_din=1, s_right_din=0 --> p_dout = 1111, s_left_dout = 1, s_right_dout = 1
select=00, p_din=1101, s_left_din=1, s_right_din=0 --> p_dout = 1111, s_left_dout = 1, s_right_dout = 1

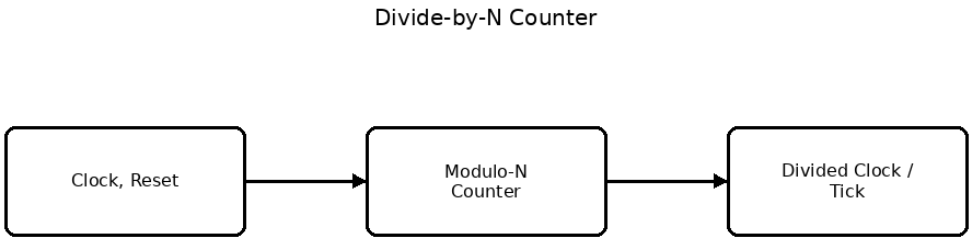
Experiment 10: Divide-by-N Counter

Design:

Truth Table / Operation Table

Parameter N	Counter sequence	Tick interval
4	0,1,2,3,0,...	Every 4 rising edges
10	0 through 9, then 0	Every 10 rising edges

Block Diagram



Verilog HDL Design Code

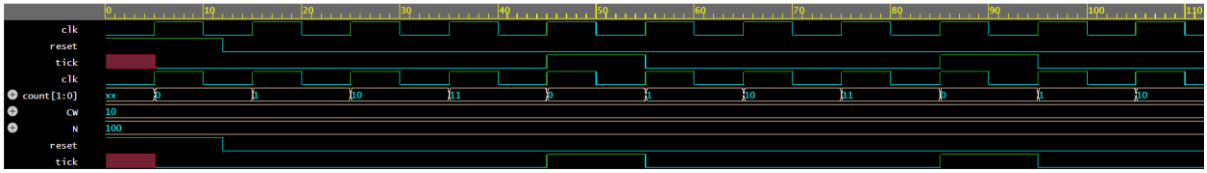
```
module divide_by_n #(
   parameter N = 4
) (
   input clk, reset,
   output reg tick
);
localparam CW = (N <= 2) ? 1 : $clog2(N);
reg [CW-1:0] count;
always @(posedge clk) begin
   if (reset) begin
      count <= 0;
      tick <= 0;
   end else if (count == N-1) begin
      count <= 0;
      tick <= 1;
   end else begin
      count <= count + 1'b1;
      tick <= 0;
   end
end
endmodule
```

Testbench Code

```
module divide_by_n_tb;
reg clk, reset;
wire tick;
divide_by_n #(4) dut(.clk(clk), .reset(reset), .tick(tick));
```

```
always #5 clk=~clk;
  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, divide_by_n_tb);
end
initial begin
    clk=0; reset=1; #12;
    reset=0; #100;
    $finish;
end
initial $monitor("t=%0t reset=%b tick=%b",$time,reset,tick);
endmodule
```

Waveforms:

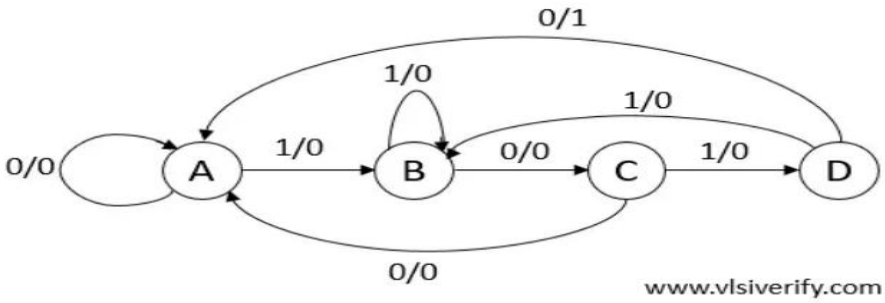


Output:

t=0 reset=1 tick=x
t=5 reset=1 tick=0
t=12 reset=0 tick=0
t=45 reset=0 tick=1
t=55 reset=0 tick=0
t=85 reset=0 tick=1
t=95 reset=0 tick=0

Experiment 11: Mealy Finite State Machine (FSM)

Design:



1010 Non-Overlapping Mealy Sequence Detector

Verilog HDL Design Code

```
module seq_detector_1010(input bit clk, rst_n, x, output z);
  parameter A = 4'h1;
  parameter B = 4'h2;
  parameter C = 4'h3;
```

```
  parameter D = 4'h4;

  bit [3:0] state, next_state;
  always @(posedge clk or negedge rst_n) begin
    if(!rst_n) begin
      state <= A;
    end
    else state <= next_state;
  end

  always @(state or x) begin
    case(state)
      A: begin
           if(x == 0) next_state = A;
           else       next_state = B;
         end
      B: begin
           if(x == 0) next_state = C;
           else       next_state = B;
         end
      C: begin
           if(x == 0) next_state = A;
           else       next_state = D;
         end
      D: begin
        if(x == 0) next_state = A; //This state only differs when compared with Mealy
Overlaping Machine
           else       next_state = B;
         end
      default: next_state = A;
    endcase
  end
  assign z = (state == D) && (x == 0)? 1:0;
endmodule
```

Testbench Code

```
 module TB;
  reg clk, rst_n, x;
  wire z;

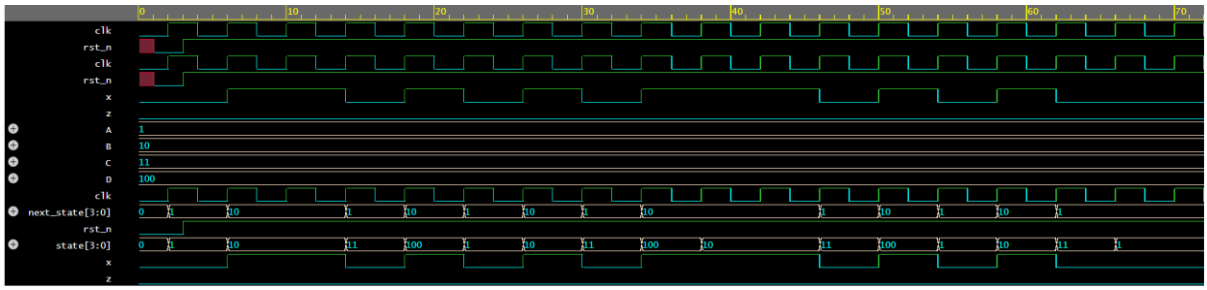
  seq_detector_1010 sd(clk, rst_n, x, z);
  initial clk = 0;
  always #2 clk = ~clk;

  initial begin
    x = 0;
    #1 rst_n = 0;
    #2 rst_n = 1;
```

```
    #3 x = 1;
    #4 x = 1;
    #4 x = 0;
    #4 x = 1;
    #4 x = 0;
    #4 x = 1;
    #4 x = 0;
    #4 x = 1;
    #4 x = 1;
    #4 x = 1;
    #4 x = 0;
    #4 x = 1;
    #4 x = 0;
    #4 x = 1;
    #4 x = 0;
    #10;
    $finish;
  end

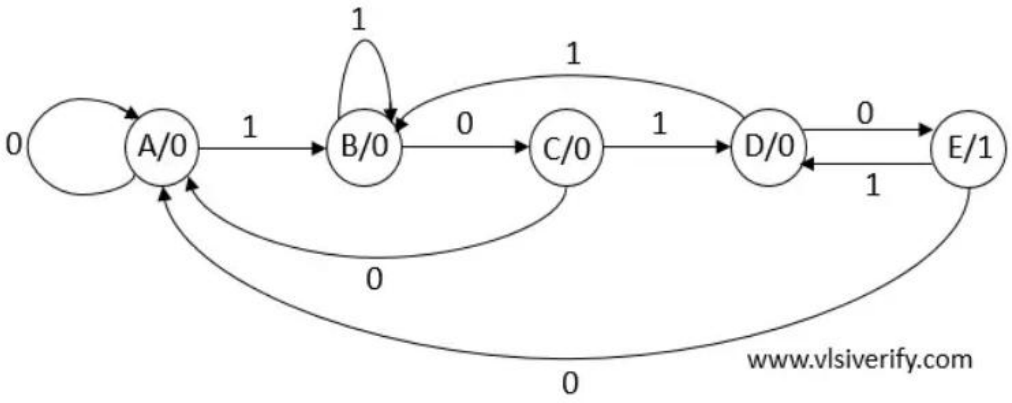
  initial begin
    // Dump waves
    $dumpfile("dump.vcd");
    $dumpvars(0, TB);
  end
endmodule
```

Waveform:



Experiment 12: Moore Finite State Machine (FSM)

Design:



1010 Overlapping Moore Sequence Detector

Verilog HDL Design Code:

```
module seq_detector_1010(input bit clk, rst_n, x, output reg z);
  parameter A = 4'h1;
  parameter B = 4'h2;
  parameter C = 4'h3;
  parameter D = 4'h4;
  parameter E = 4'h5; // extra state when compared with Mealy Machine

  bit [3:0] state, next_state;
  always @(posedge clk or negedge rst_n) begin
    if(!rst_n) begin
      state <= A;
    end
    else state <= next_state;
  end

  always @(state or x) begin
    case(state)
      A: begin
           if(x == 0) next_state = A;
           else       next_state = B;
         end
      B: begin
           if(x == 0) next_state = C;
           else       next_state = B;
         end
      C: begin
           if(x == 0) next_state = A;
           else       next_state = D;
         end
      D: begin
```

```
        if(x == 0) next_state = E;
        else       next_state = B;
      end
    E: begin
        if(x == 0) next_state = A;
        else       next_state = D;
      end
    default: next_state = A;
  endcase
 end
 //As output z is only depends on present state
 always@(state) begin
  case(state)
    A : z = 0;
    B : z = 0;
    C : z = 0;
    D : z = 0;
    E : z = 1;
    default : z = 0;
  endcase
 end
endmodule
```

Testbench Code:

```
module TB;
 reg clk, rst_n, x;
 wire z;

 seq_detector_1010 sd(clk, rst_n, x, z);
 initial clk = 0;
 always #2 clk = ~clk;

 initial begin
  x = 0;
  #1 rst_n = 0;
  #2 rst_n = 1;

  #3 x = 1;
  #4 x = 1;
  #4 x = 0;
  #4 x = 1;
  #4 x = 0;
  #4 x = 1;
  #4 x = 0;
  #4 x = 1;
  #4 x = 1;
  #4 x = 1;
  #4 x = 0;
  #4 x = 1;
```

```
    #4 x = 0;
    #4 x = 1;
    #4 x = 0;
    #10;
    $finish;
  end

  initial begin
    // Dump waves
    $dumpfile("dump.vcd");
    $dumpvars(0, TB);
  end
endmodule
```

Waveforms:

